VSB TECHNICAL UNIVERSITY OF OSTRAVA | FACULTY OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Trie Data Structure
Datová Struktura Trie

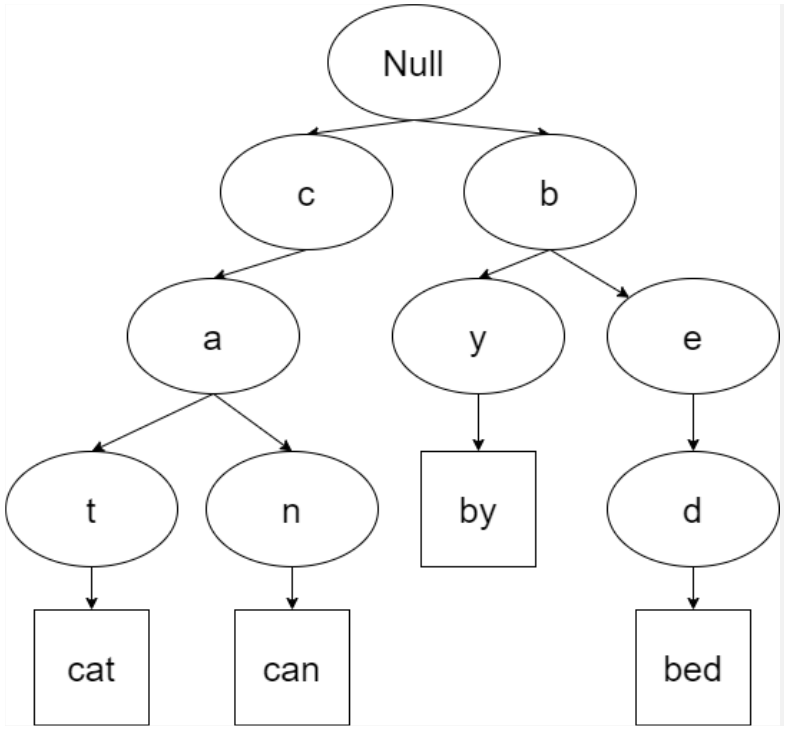
Author: Tuan Phong HUYNH

Supervisor: Prof. Ing. Michal Krátký, Ph.D.

Year: 2025

Introduction – What is a Trie?

- Known as a digital tree or prefix tree.
- Used in autocomplete, spell-checking…
- Searching time proportional to word length, not data size.
- Challenging to implement efficiently for large alphabets (Unicode)



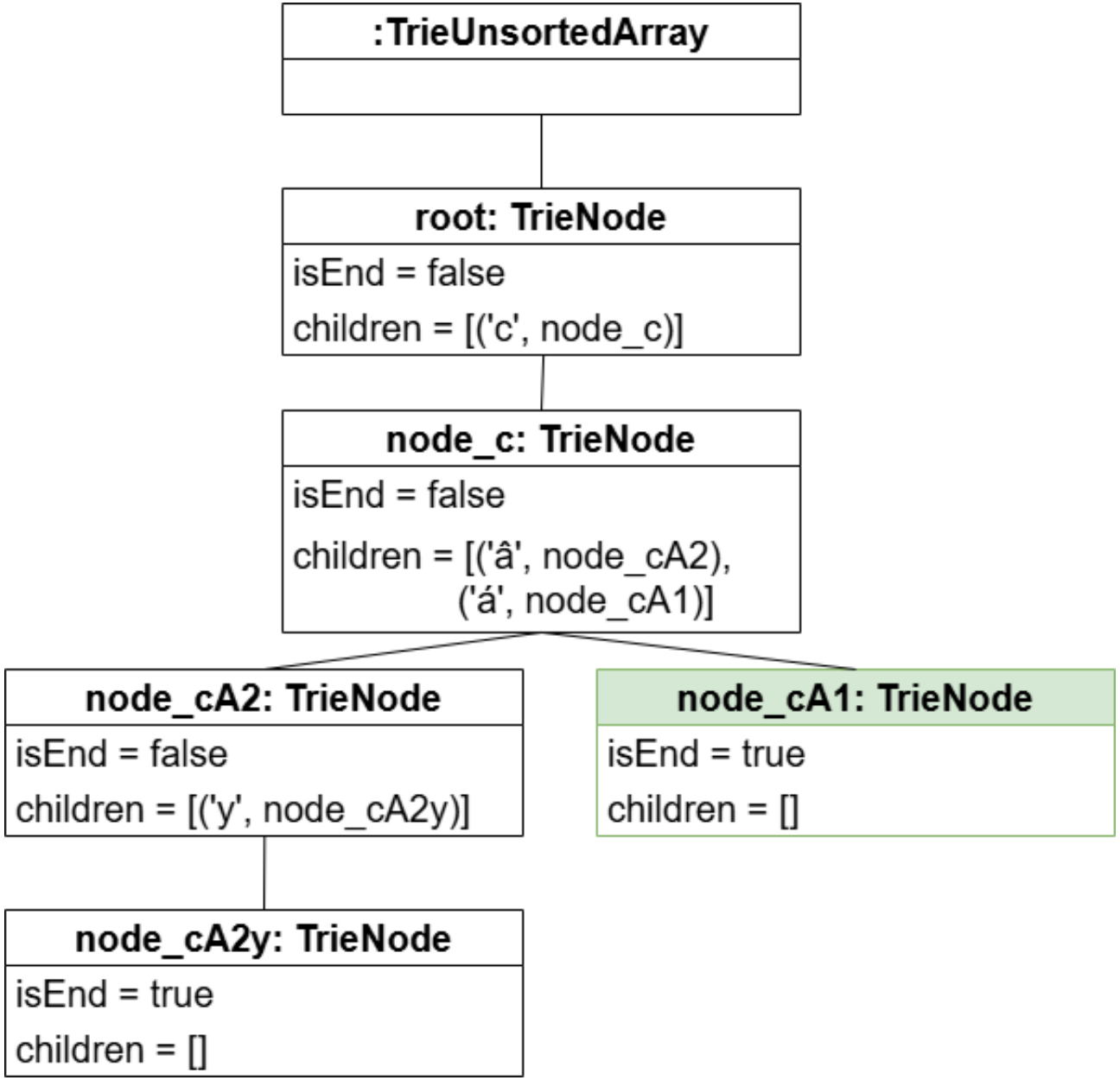
Objective of the Thesis

- Analyze and compare different Trie data structures:
  - Unsorted Trie
  - Sorted Trie using Binary Search
  - Hash Table Trie using Linear Probing
- Focus on performance of 3 basic operations with large alphabet:
  - Insert
  - Search
  - Delete
- Compare results with:
  - Sequential Array
  - Compressed Trie
  - Hash Table Trie using Quadratic Probing

Node implementation: unsorted Array

$|C|$: The actual number of children in a node

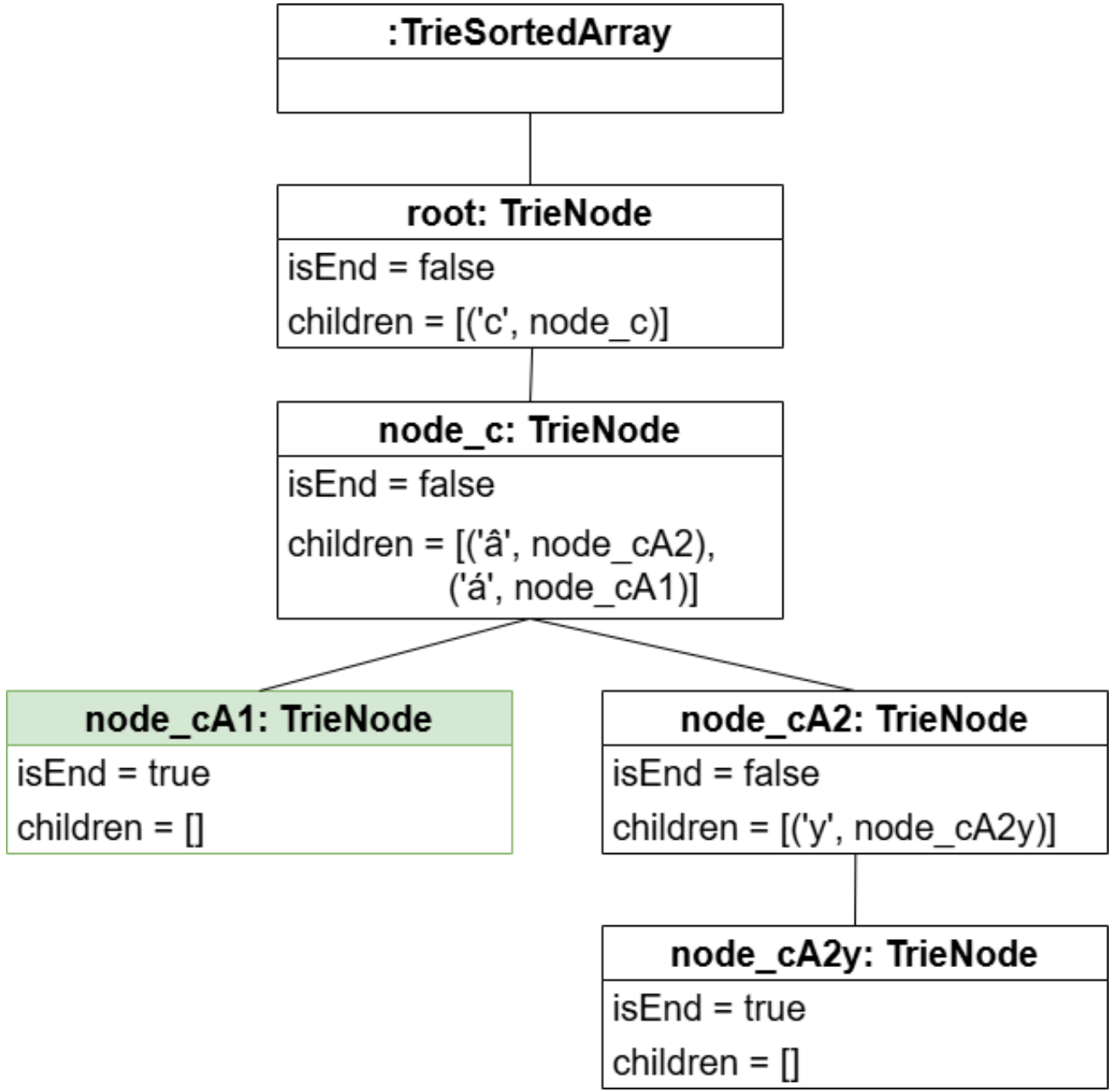
- Insert: O($|C|$)
- Search: O($|C|$)
- Delete: O($|C|$)



Node implementation: sorted Array

$|C|$: The actual number of children in a node

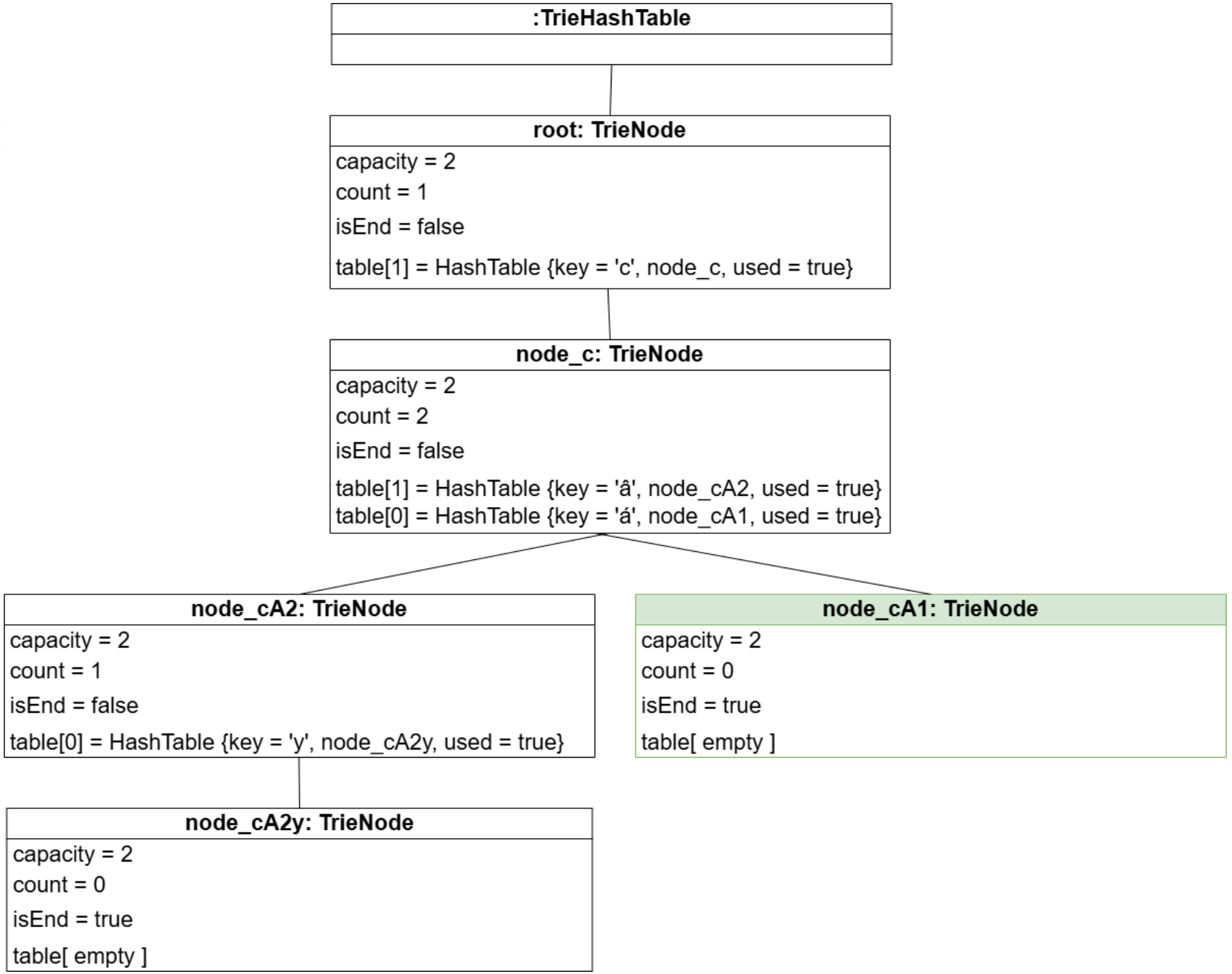
- Insert: O($|C|$) $\rightarrow$ O(log$|C|$)
- Search: O(log$|C|$)
- Delete: O(log$|C|$)



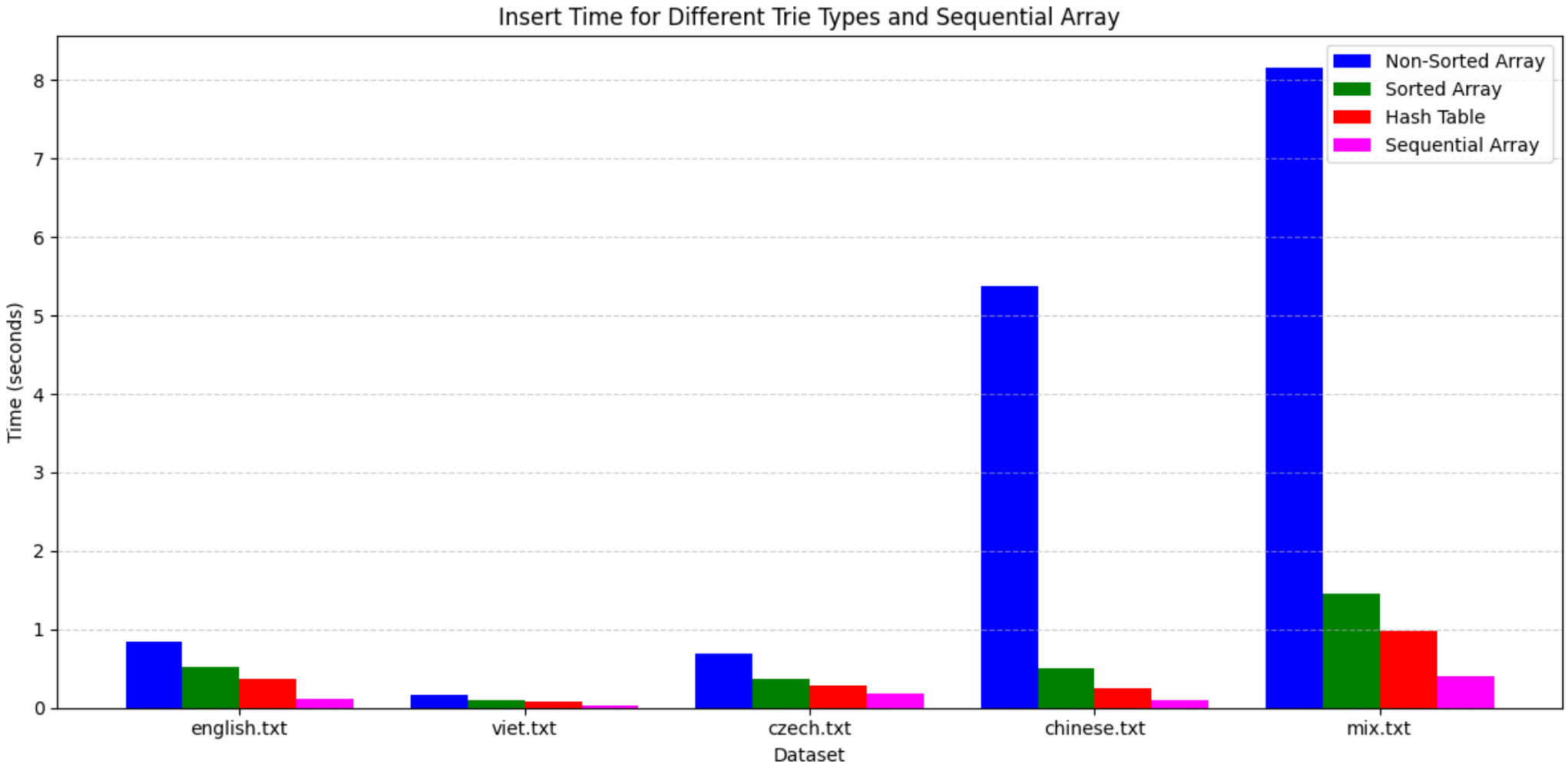
Node implementation: Hash Table

$|C|$: The actual number of children in a node

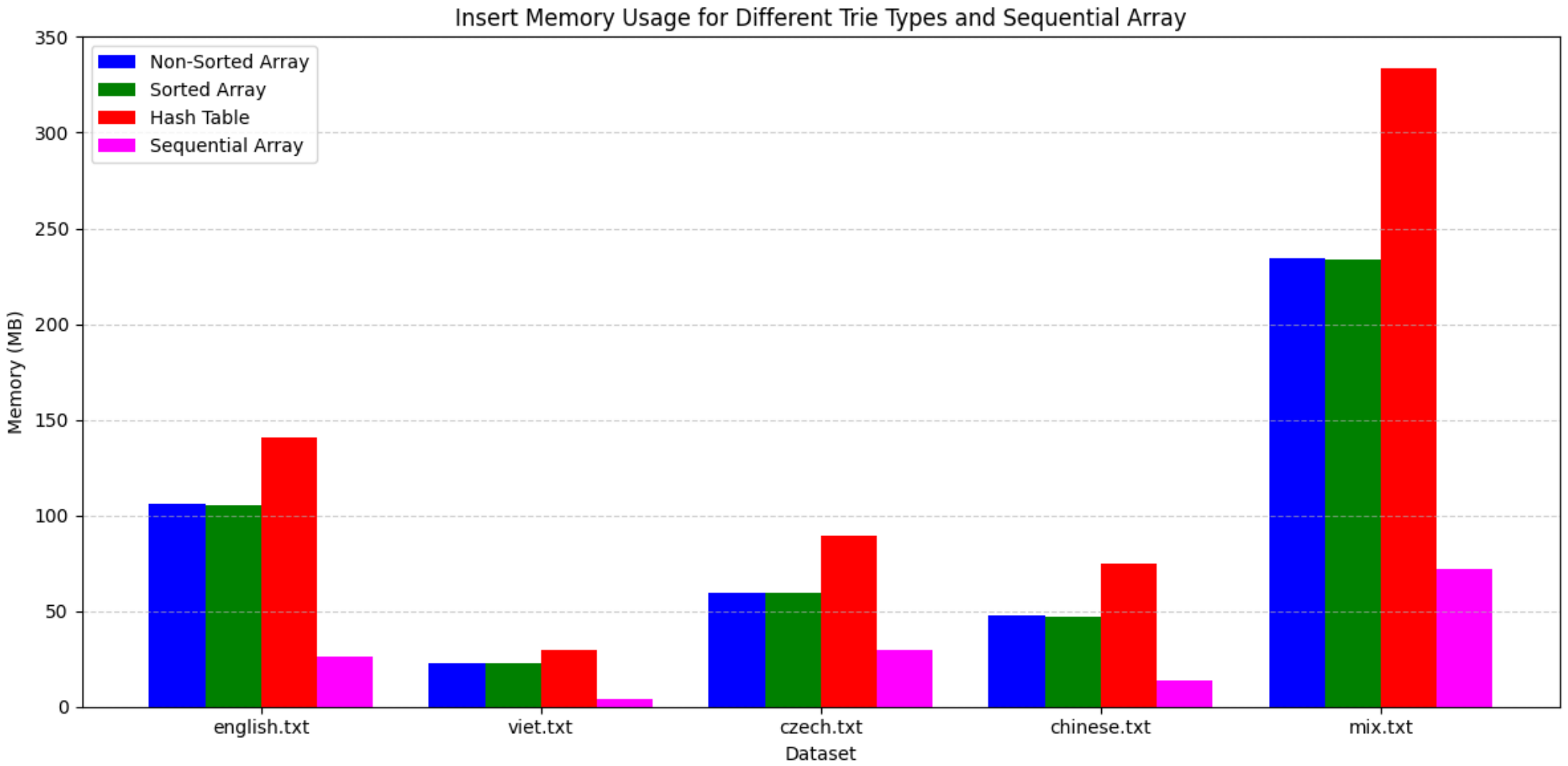
- Insert: O(1)
  - Worse case: O($|C|$)
- Search: O(1)
- Delete: O(1)



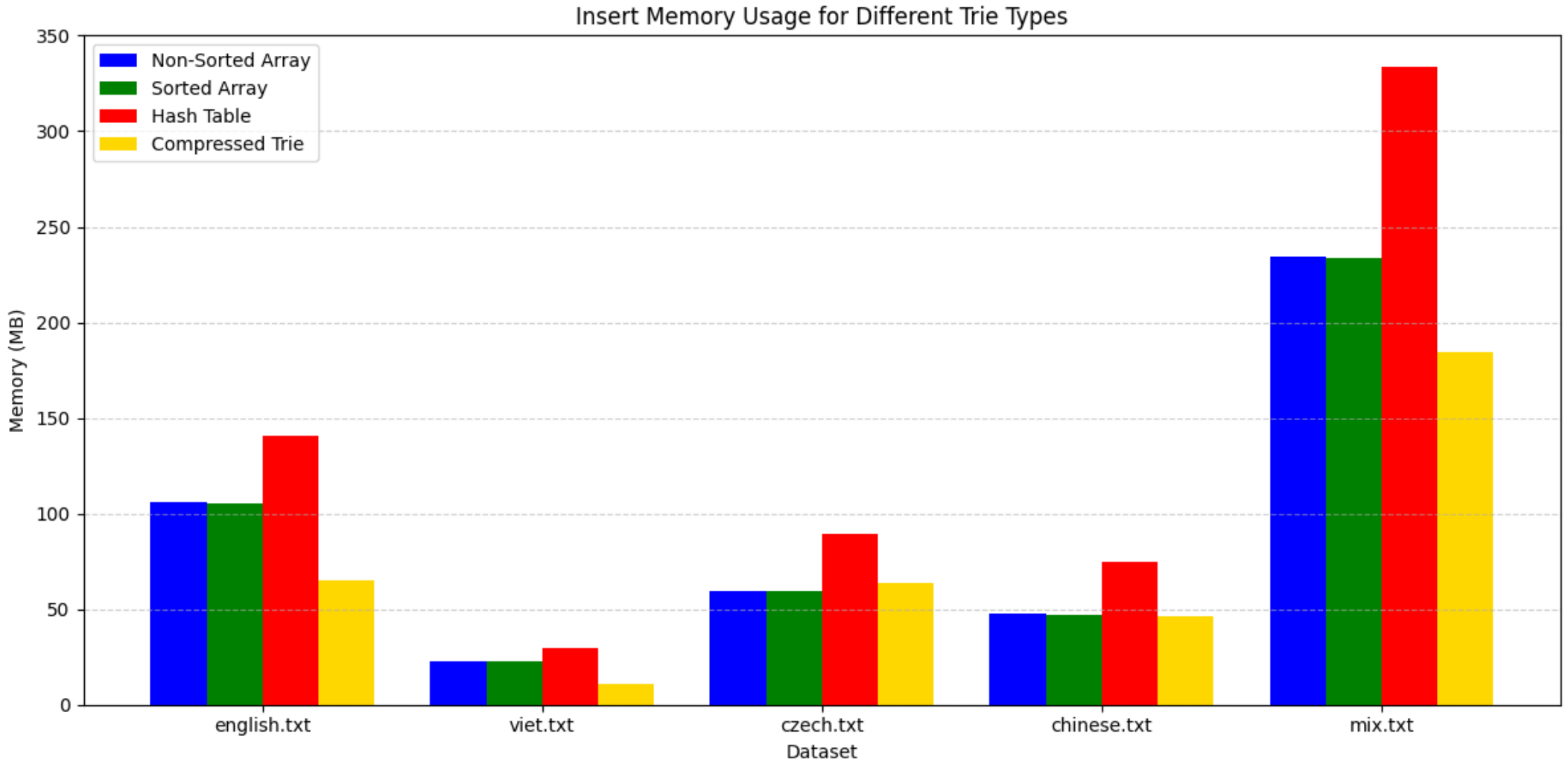
Experiment: Insert – Time



Experiment: Insert – Memory



Experiment: Insert of Compressed Trie – Memory



Experiment: Search – Time

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
english.txt	10,000	0.019	0.013	0.011	35.140
viet.txt	10,000	0.019	0.010	0.009	4.770
czech.txt	10,000	0.016	0.012	0.010	42.261
chinese.txt	10,000	0.132	0.007	0.005	35.109
mix.txt	10,000	0.056	0.013	0.011	105.800

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
english.txt	pne	0.00032	0.00029	0.00049	0.01242
viet.txt	cái	0.00013	0.00012	0.00022	0.00200
czech.txt	nez	0.00104	0.00101	0.00183	0.01404
chinese.txt	人	0.00061	0.00058	0.00116	0.00796
mix.txt	in	0.01114	0.01111	0.02088	0.03673

Experiment: Search of Linear Probing and Quadratic Probing – Time

Dataset	Prefix	Hash Table (Linear)	Hash Table (Quadratic)
`english.txt`	pne	4.90×10^{-4}	2.50×10^{-4}
`viet.txt`	cái	2.20×10^{-4}	1.60×10^{-6}
`czech.txt`	nez	1.83×10^{-3}	1.03×10^{-3}
`chinese.txt`	人	1.16×10^{-3}	7.00×10^{-7}
`mix.txt`	in	2.09×10^{-2}	9.63×10^{-3}

Experiment: Delete – Time

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
english.txt	10,000	0.030	0.021	0.017	232.709
viet.txt	10,000	0.035	0.019	0.015	32.196
czech.txt	10,000	0.026	0.017	0.015	274.490
chinese.txt	10,000	0.245	0.016	0.008	218.652
mix.txt	10,000	0.104	0.021	0.017	735.939

Conclusion

- Hash Table Trie is fastest → uses more memory
- Sequential Array saves space → slow
- Sorted Array offers a good balance of speed and memory.
- Unsorted Array is simple to implement → Less effiency for large alphabets.
- Trie is confirmed as an efficient structure for string processing

Thank You

Defense Answers

1. Encoding of unicode characters in nodes is not clear. Perhaps, it can be hidden using codecvt, but it is rather important to get the optimal size of the Trie. Is it encoded using the fixed-size, 4B, values? Or using the variable-size, 1-4B, values?
   - **Fixed-size encoding:** 4 bytes → good for using variable length code in future works
2. Why the number of nodes with 0 children dominate in the case of Czech and Chinese, whereas it is 1 for Vietnamese and English (see page 33)?
   - **Datasets contain many unique and short words:**
   - Chinese – single word is a full word, Czech – many short variants for a word
   - English – Vietnamese: long prefixes
3. It is not clear why the results of Non-sorted array are always worst although you show that the majority of nodes includes only one value (see page 33).
   - **Small nodes with a large number of children** ($|C| >> 1$) – near root
   - Definitely appear in search → Linearly search → making results worst
   - Sorted Array (binary search) and Hash Table (hash function) → index → directly access

Defense Answers (cont.)

1. In the case of the Sorted array node, you propose 0.013s for 10 000 queries over the mix data collection, in the case of the Sequential array it is 105.8s. It is not clear why it is 0.011 and 0.036 for prefix searching, i.e. the Sorted array node is only 3x faster?
   - **Because prefix search different from a whole string search:** A whole string – following 1 path. Prefix search – traverse all words under that node → recursive traversal a lots.
   - **Recursive same for all**. Advantage of binary search is less for searching 1-2 nodes. Sequential array: check prefix – add into array – move to next string.
2. Can you compare your work with the article from the assignment: "Aoe, J.-I., Morimoto, K. and Sato, T. (1992), An efficient implementation of trie structures. Softw: Pract. Exper., 22: 695-721. https://doi.org/10.1002/spe.4380220902"?
   - **Article work:** Double-array, tail compression, fast access, memory efficiency, ASCII only.
   - **My work:** Array with pair <character, node>, various structures, not memory effciency, Unicode.
3. Would it be possible to avoid the result materialization in the prefix searching using an iterator?
   - **Iterator for searching prefix:** save memory, slowdown searching, good for searching few first strings.